Report for Assignment 4 Question 2: Part B

Nur Alya Maisarah Abd Karim, A00268252

Contents

---

1 Introduction

This report aims to illustrate how R Studio can be used effectively for cleaning, transforming, and analyzing property datasets. Analyzing property data is essential for understanding market trends, price fluctuations, and factors influencing real estate decisions. R Studio offers a robust set of tools and packages that support data management, cleaning, and visualization. The focus of this report is on two property datasets, "PropertyDataset1.csv" and "PropertyDataset2.csv," which contain valuable property-related information. The report covers key tasks such as importing the data, cleaning and preprocessing it, creating new variables, and visualizing insights to uncover patterns in the property market.

2 Challenges

Working with property datasets often presents challenges due to the inherent complexity of real estate data. One common issue is missing values, which can occur in fields like property prices, descriptions, or location details. These gaps must be addressed properly to avoid impacting the accuracy of any analysis. Dates in property data are also often formatted inconsistently, making it difficult to standardize them for analysis.

Unstructured data, such as property descriptions, is another challenge. These free-text fields must be categorized into meaningful variables, such as property type or condition, which requires the application of text analysis techniques.

Inconsistent data formats, particularly with numeric fields like prices, also pose problems. Price formats may vary due to currency symbols, commas, or regional differences, which can lead to discrepancies. Similarly, location data may have different naming conventions, complicating geographic analysis and data integration. Such challenges emphasize the importance of thorough data cleaning and transformation.

Example

To better understand how inconsistent data formats or missing values may appear, you can inspect the first few rows of the dataset:

```
# Load necessary libraries
library(dplyr)
library(ggplot2)
library(tidyr)
library(tidyselect)
library(lubridate)
library(stringr)
library(tinytex)

# Load the datasets
propertydataset1part2b <- read.csv("PropertyDataset1.csv")
propertydataset2part2b <- read.csv("PropertyDataset2.csv",
                                   fileEncoding = "ISO-8859-1")

# View the first few rows of the dataset
head(propertydataset1part2b)
head(propertydataset2part2b)

# Check the data types of each column
str(propertydataset1part2b)
str(propertydataset2part2b)
```

In this example:

The head() function helps reveal missing values, inconsistencies, or incorrect formats. The str() function identifies column data types, ensuring the data is correctly formatted.

3 Data Import and Initial Inspection

When working with property datasets, the first step is importing the data into R Studio. The read.csv() function is typically used for CSV files, while Excel files can be imported using the read_excel() function from the readxl package. These functions allow seamless loading of data into R for further analysis. For instance, to load a CSV file:

```
# Load necessary libraries
library(dplyr)
library(ggplot2)
library(tidyr)
library(tidyselect)
library(lubridate)
library(stringr)
library(tinytex)

# Load the datasets
dataset1part2b <- read.csv("PropertyDataset1.csv")
dataset2part2b <- read.csv("PropertyDataset2.csv",fileEncoding = "ISO-8859-1")
```

After importing the data, it's essential to explore the dataset to understand its structure and identify any potential issues. This can be done using several functions:

str(): Provides a summary of the datasets structure, including the data types of each column. This is particularly useful to identify if columns are incorrectly formatted. For example, if a sale_date column is being treated as a character type instead of a Date, we can correct it.

```
# Check structure of the dataset
str(dataset1part2b)
str(dataset2part2b)
```

summary(): Summarizes the statistical properties of the dataset, such as the range, mean, and any missing values in the columns. This is useful for spotting outliers or gaps in the data.

```
# Get a summary of the dataset
summary(dataset1part2b)
```

```
##    geohash           sale_date             price          formatted_address
##  Length:40000       Length:40000       Min.   :       0   Length:40000
##  Class :character   Class :character   1st Qu.:  115400   Class :character
##  Mode  :character   Mode  :character   Median :  195000   Mode  :character
##                                        Mean   :  254256
##                                        3rd Qu.:  300000
##                                        Max.   :69873482
##  raw_address         post_code             county          num_bedrooms
##  Length:40000       Length:40000       Length:40000       Length:40000
##  Class :character   Class :character   Class :character   Class :character
```

```
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##  num_bathrooms      property_description property_size_description
##  Length:40000       Length:40000         Length:40000
##  Class :character   Class :character     Class :character
##  Mode  :character   Mode  :character     Mode  :character
##
##
##
##  formatted_description not_full_market_price vat_exclusive
##  Length:40000          Length:40000          Length:40000
##  Class :character      Class :character      Class :character
##  Mode  :character      Mode  :character      Mode  :character
##
##
##
##    parking            parking1
##  Length:40000       Length:40000
##  Class :character   Class :character
##  Mode  :character   Mode  :character
##
##
##
```

```
summary(dataset2part2b)
```

```
##  Date.of.Sale..dd.mm.yyyy.   Address             County
##  Length:628034              Length:628034      Length:628034
##  Class :character           Class :character   Class :character
##  Mode  :character           Mode  :character   Mode  :character
##   Price....          Not.Full.Market.Price VAT.Exclusive
##  Length:628034      Length:628034         Length:628034
##  Class :character   Class :character      Class :character
##  Mode  :character   Mode  :character      Mode  :character
##  Description.of.Property Property.Size.Description
##  Length:628034           Length:628034
##  Class :character        Class :character
##  Mode  :character        Mode  :character
```

head(): Displays the first few rows of the dataset, allowing a quick preview of the data. This can help identify any immediate issues, like missing or incorrect values.

```
# View the first few rows
head(dataset1part2b)
```

```
##        geohash  sale_date     price
## 1              29/05/2020  63436.12
## 2 gce8t0tbed4u 17/12/2019 341000.00
## 3 gc6g7r445n7p 09/02/2017 170000.00
## 4 gc7j3n5rvwmc 19/12/2018 112000.00
## 5 gc65bu202d1y 31/03/2017 173000.00
```

```
## 6 gc7rb1tmkzz8 08/03/2019 405000.00
##                                 formatted_address
## 1
## 2                4 Woodland Park, Rush, Co. Dublin
## 3 50 Marble Crest, Kilkenny, Kilkenny, Co. Kilkenny
## 4       18 Marina Court, Athy, Kildare, Co. Kildare
## 5              61 Rosehill, Newport, Co. Tipperary
## 6
##                                   raw_address post_code    county
## 1 APARTMENT NO. 7, LIBRARY CORNER, MANORHAMILTON             Leitrim
## 2                  4 WOODLAND PARK, RUSH, DUBLIN              Dublin
## 3            50 MARBLE CREST, KILKENNY, KILKENNY            Kilkenny
## 4                18 MARINA COURT, ATHY, KILDARE             Kildare
## 5             61 ROSEHILL, NEWPORT, CO TIPPERARY           Tipperary
## 6        22 Fenton Green, Church Street, Kilcock             Kildare
##   num_bedrooms num_bathrooms                  property_description
## 1                                    New Dwelling house /Apartment
## 2   3 Bedrooms   2 Bathrooms Second-Hand Dwelling house /Apartment
## 3                            Second-Hand Dwelling house /Apartment
## 4                            Second-Hand Dwelling house /Apartment
## 5   4 Bedrooms   3 Bathrooms Second-Hand Dwelling house /Apartment
## 6                                    New Dwelling house /Apartment
##   property_size_description                formatted_description
## 1
## 2                                            Semi-Detached House
## 3                           Second-Hand Dwelling House/Apartment
## 4                           Second-Hand Dwelling House/Apartment
## 5                                            Semi-Detached House
## 6
##   not_full_market_price vat_exclusive parking parking1
## 1                 FALSE          TRUE     Has  Parking
## 2                 FALSE         FALSE     Has  Parking
## 3                 FALSE         FALSE     Has  Parking
## 4                 FALSE         FALSE     Has  Parking
## 5                 FALSE         FALSE     Has  Parking
## 6                 FALSE          TRUE     Has  Parking
```

```
head(dataset2part2b)
```

```
##   Date.of.Sale..dd.mm.yyyy.                                        Address
## 1                01/01/2010         5 Braemor Drive, Churchtown, Co.Dublin
## 2                03/01/2010 134 Ashewood Walk, Summerhill Lane, Portlaoise
## 3                04/01/2010            1 Meadow Avenue, Dundrum, Dublin 14
## 4                04/01/2010                        1 The Haven, Mornington
## 5                04/01/2010                  11 Melville Heights, Kilkenny
## 6                04/01/2010                 12 Sallymount Avenue, Ranelagh
##     County        Price.... Not.Full.Market.Price VAT.Exclusive
## 1   Dublin \u0080343,000.00                    No            No
## 2    Laois \u0080185,000.00                    No           Yes
## 3   Dublin \u0080438,500.00                    No            No
## 4    Meath \u0080400,000.00                    No            No
## 5 Kilkenny \u0080160,000.00                    No            No
## 6   Dublin \u0080425,000.00                    No            No
##                Description.of.Property
```

```
## 1 Second-Hand Dwelling house /Apartment
## 2         New Dwelling house /Apartment
## 3 Second-Hand Dwelling house /Apartment
## 4 Second-Hand Dwelling house /Apartment
## 5 Second-Hand Dwelling house /Apartment
## 6 Second-Hand Dwelling house /Apartment
##                                           Property.Size.Description
## 1
## 2 greater than or equal to 38 sq metres and less than 125 sq metres
## 3
## 4
## 5
## 6
```

4 Cleaning the Data

Data cleaning is a crucial step in the data analysis workflow, ensuring that the datasets are consistent, accurate, and free from errors or inconsistencies. Without cleaning, incorrect or incomplete data can lead to misleading results and faulty conclusions. The R code snippet below demonstrates the essential process of cleaning and standardizing two property-related datasets—**PropertyDataset1** and **PropertyDataset2**—for further analysis.

Code Breakdown:

1. **Standardizing Column Names**: Both datasets have column names that need to be standardized to ensure consistency. This step helps avoid errors during merging and makes the data easier to work with by aligning columns such as `price`, `sale_date`, and `county` to consistent names across both datasets.
2. **Handling Missing Values**: Missing data is addressed by using the `na.omit()` function, which removes rows containing any missing values. This approach helps prevent errors during analysis, although it reduces the number of rows. Alternatively, missing data could be imputed, replacing missing values with statistical estimates like the mean or median.
3. **Cleaning Date and Price Variables**: The `sale_date` field is standardized using `lubridate`'s `dmy()` function to ensure consistent date formatting. The `price` field is cleaned by removing currency symbols and commas, followed by converting the data to numeric values for accurate analysis.
4. **Merging the Datasets**: After cleaning, both datasets are combined into a single dataset using `bind_rows()`, making it ready for further analysis.

Finally, the structure of the combined dataset is checked using `str()` and summary statistics are generated with `summary()`.

```r
# Standardize column names for consistency
colnames(dataset1part2b) <- c("geohash", "sale_date", "price",
                              "formatted_address", "raw_address",
                                "post_code", "county",
                              "num_bedrooms", "num_bathrooms",
                                "property_description",
                              "property_size_description",
                                "formatted_description",
                              "not_full_market_price",
                                "vat_exclusive", "parking", "parking1")
```

```r
colnames(dataset2part2b) <- c("sale_date", "address", "county",
                              "price", "not_full_market_price",
                                "vat_exclusive", "property_description",
                              "property_size_description")

# Replace blank ("") values with NA for dataset1part2b
dataset1part2b[dataset1part2b == ""] <- NA

# Replace blank ("") values with NA for dataset2part2b
dataset2part2b[dataset2part2b == ""] <- NA

# Remove rows with missing values (NA) for both datasets
dataset1part2b_clean <- dataset1part2b
dataset2part2b_clean <- dataset2part2b

# Standardize date format (assuming it's in dd/mm/yyyy format in both datasets)
dataset1part2b_clean$sale_date <- dmy(dataset1part2b_clean$sale_date)
dataset2part2b_clean$sale_date <- dmy(dataset2part2b_clean$sale_date)
```

```
## Warning: 3 failed to parse.
```

```r
# Remove currency symbols and commas
dataset2part2b_clean$price <- gsub("[^0-9.]", "", dataset2part2b_clean$price)

# Clean price (remove currency symbols and commas)
dataset1part2b_clean$price <- as.numeric(gsub("€|,", "", dataset1part2b_clean$price))
dataset2part2b_clean$price <- as.numeric(gsub("€|,", "", dataset2part2b_clean$price))

# Combine the cleaned datasets
combined_data_2b <- bind_rows(dataset1part2b_clean, dataset2part2b_clean)

# Inspect combined dataset structure and summary statistics
str(combined_data_2b)
summary(combined_data_2b$price)
```

5 Removing Outliers

Removing outliers is an essential part of data cleaning, as extreme values can skew analysis and lead to misleading conclusions. Outliers may arise from data entry mistakes or may represent rare, but valid, occurrences. Identifying and addressing these outliers ensures that the dataset more accurately reflects the general pattern of the data, improving the validity of any analysis performed. A widely used method to detect outliers is the Interquartile Range (IQR) technique. The IQR measures the spread of the middle 50% of the data by subtracting the first quartile (Q1) from the third quartile (Q3). Any data points that fall outside the bounds of Q1 - 1.5 * IQR or Q3 + 1.5 * IQR are considered outliers.

In R, the `filter()` function is commonly used to remove outliers that fall outside these bounds, resulting in a cleaner dataset that is more representative of the overall data distribution. For instance, in the `combined_data_2b` dataset, a custom function called `remove_outliers` can be applied to the `price` column. This function first calculates the IQR, defines the upper and lower bounds, and then filters out any data points that fall outside these bounds. The cleaned dataset, `combined_data_2b_clean`, now contains the `price` column without the influence of extreme outliers. This process helps ensure that subsequent analyses reflect a more accurate picture of the data, free from the distortions caused by extreme values.

```r
# Function to remove outliers based on IQR
remove_outliers <- function(data, column) {
  # Ensure the column is numeric
  if (!is.numeric(data[[column]])) {
    stop(paste("Column", column, "must be numeric."))
  }

  # Calculate the IQR
  Q1 <- quantile(data[[column]], 0.25, na.rm = TRUE)
  Q3 <- quantile(data[[column]], 0.75, na.rm = TRUE)
  IQR_value <- Q3 - Q1

  # Calculate bounds
  lower_bound <- Q1 - 1.5 * IQR_value
  upper_bound <- Q3 + 1.5 * IQR_value

  # Filter data
  data_clean <- data %>% filter(data[[column]] >= lower_bound & data[[column]] <= upper_bound)

  return(data_clean)
}

# Ensure numeric columns
combined_data_2b$price <- as.numeric(gsub("[^0-9.]", "", combined_data_2b$price))
combined_data_2b <- combined_data_2b[!is.na(combined_data_2b$price), ]

combined_data_2b$sale_date <- as.Date(combined_data_2b$sale_date, format = "%d/%m/%Y")
combined_data_2b$year <- as.numeric(format(combined_data_2b$sale_date, "%Y"))
combined_data_2b <- combined_data_2b[!is.na(combined_data_2b$year), ]

# Remove outliers from 'price'
combined_data_2b_clean <- remove_outliers(combined_data_2b, "price")

# Remove outliers from 'year'
combined_data_2b_clean <- remove_outliers(combined_data_2b_clean, "year")

# Inspect cleaned data
summary(combined_data_2b_clean$price)
summary(combined_data_2b_clean$year)
```

6 Additional Variables

Creating additional variables enhances the dataset, providing more dimensions for analysis and helping to uncover hidden patterns. For example, the extraction of sale_year, sale_month, and sale_day from the sale_date column allows the data to be analyzed by specific time periods, which is useful for identifying seasonal trends in property prices. Another important feature, price_category, classifies properties into "High" or "Low" price categories based on the median price, helping to identify pricing trends and facilitate comparison.

The description_length variable measures the length of the property description, offering insight into how detailed or concise listings are, which could influence buyer interest. Additionally, the address_length variable provides information about the address length, which could be relevant in understanding whether

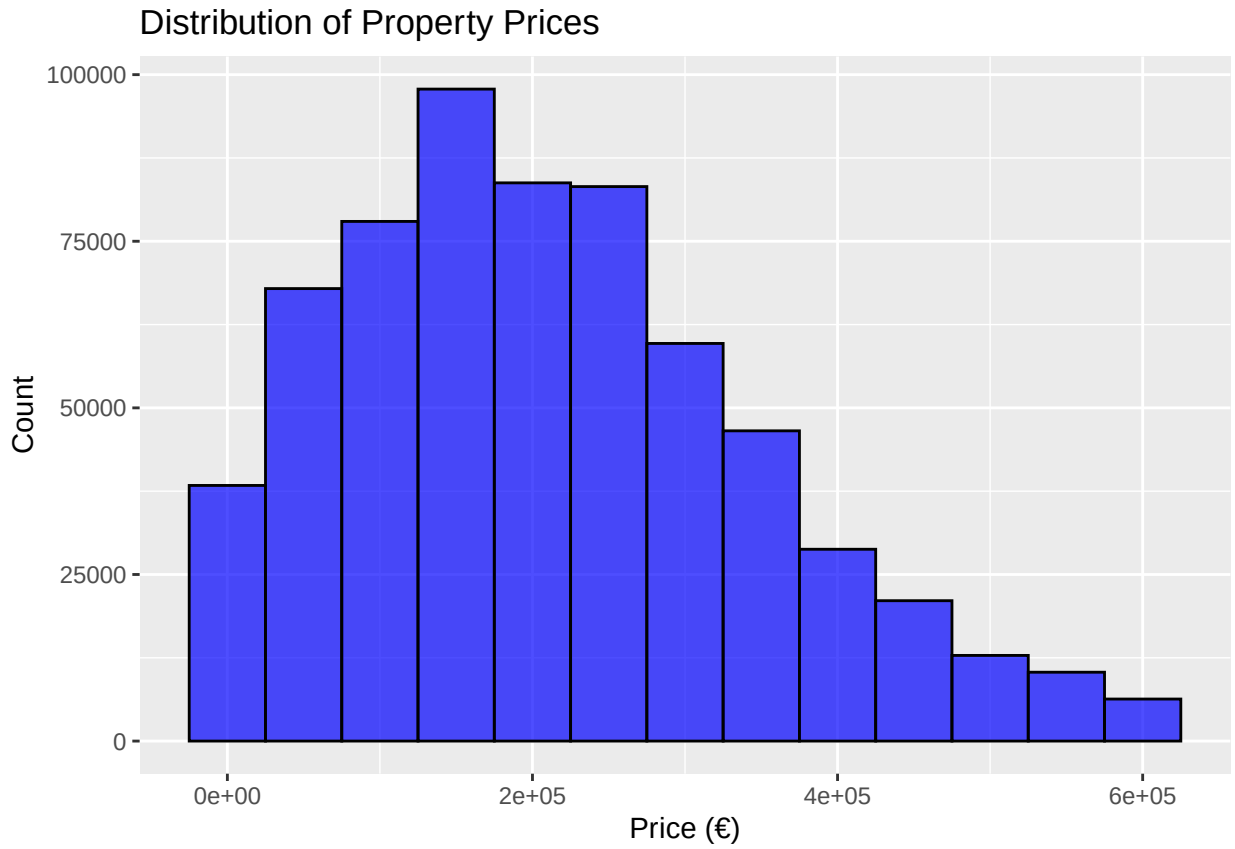
location specifics correlate with property price. Finally, the full_market_price variable helps to categorize properties as either having a full market price or not, which could be useful for understanding pricing strategies. These new variables contribute to the richness of the dataset, offering more explanatory factors for predictive modeling and analysis.

```
# Extract year, month, and day from the sale_date column
combined_data_2b_clean$sale_year <- year(combined_data_2b_clean$sale_date)
combined_data_2b_clean$sale_month <- month(combined_data_2b_clean$sale_date)
combined_data_2b_clean$sale_day <- day(combined_data_2b_clean$sale_date)

# Create a variable to categorize properties into high or low price based on median
median_price <- median(combined_data_2b_clean$price, na.rm = TRUE)
combined_data_2b_clean$price_category <-
  ifelse(combined_data_2b_clean$price > median_price, "High", "Low")

# Create a variable for the length of the property description
combined_data_2b_clean$description_length <-
  nchar(as.character(combined_data_2b_clean$property_description))

# Create a variable for the number of characters in the address
combined_data_2b_clean$address_length <-
  nchar(as.character(combined_data_2b_clean$formatted_address))

# Create a variable to indicate whether a property has a full market price or not
combined_data_2b_clean$full_market_price <-
  ifelse(combined_data_2b_clean$not_full_market_price == "No", "No", "Yes")
```

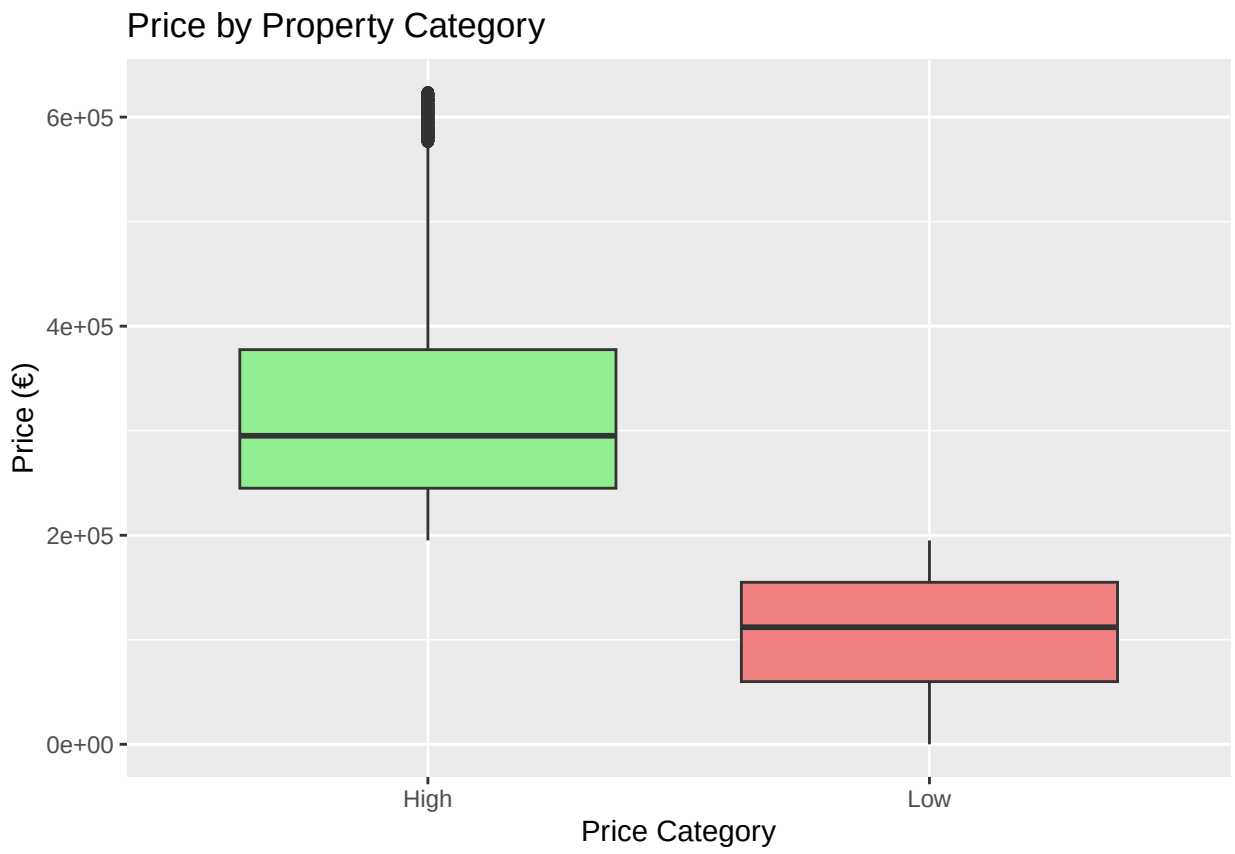
7 Data Visualization and Other Features

Data visualization plays a crucial role in uncovering patterns, trends, and anomalies in the data, helping to better understand the dataset and make informed decisions. In addition to visualizing key relationships between variables, creating new features based on insights from the data can provide a deeper understanding and improve predictive modeling.

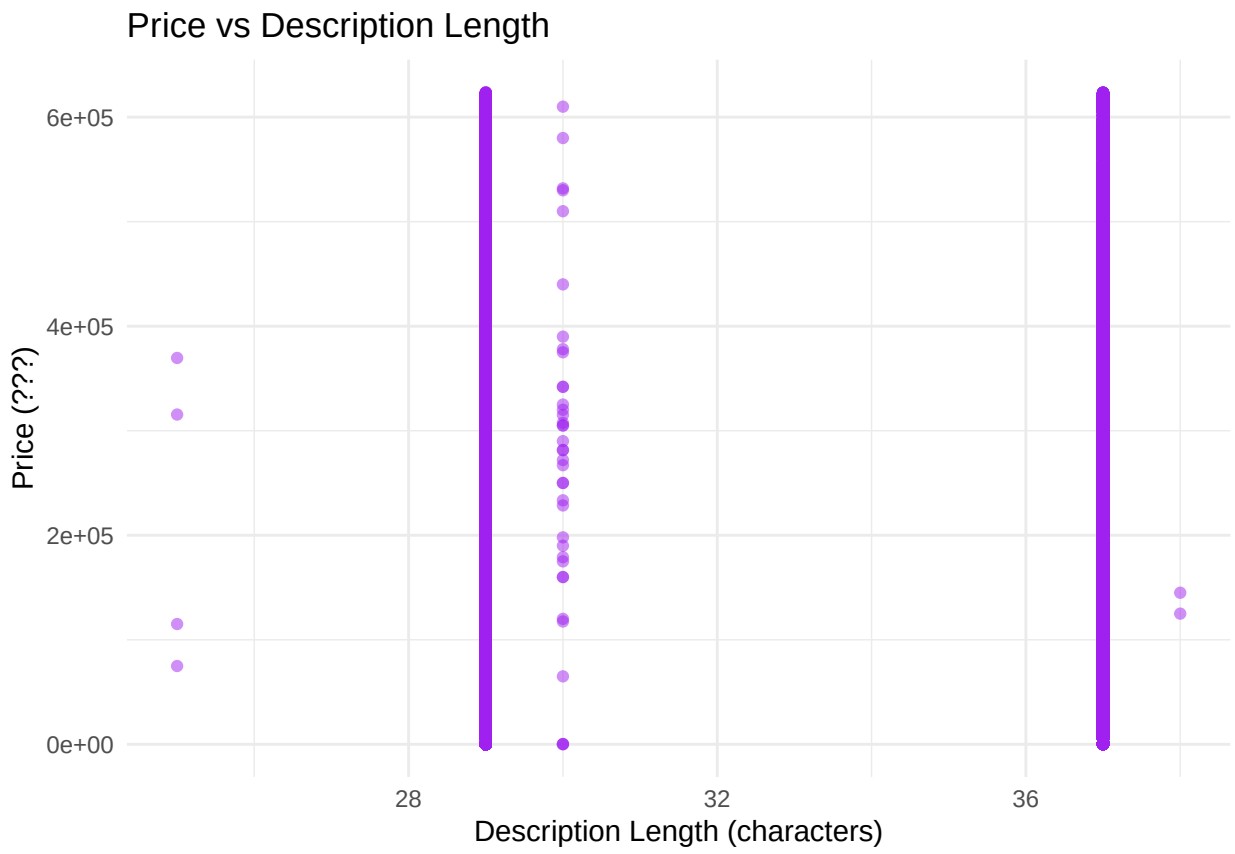
```
# Histogram of property prices
ggplot(combined_data_2b_clean, aes(x = price)) +
  geom_histogram(binwidth = 50000, fill = "blue", color = "black", alpha = 0.7) +
  labs(title = "Distribution of Property Prices", x = "Price (€)", y = "Count")
```



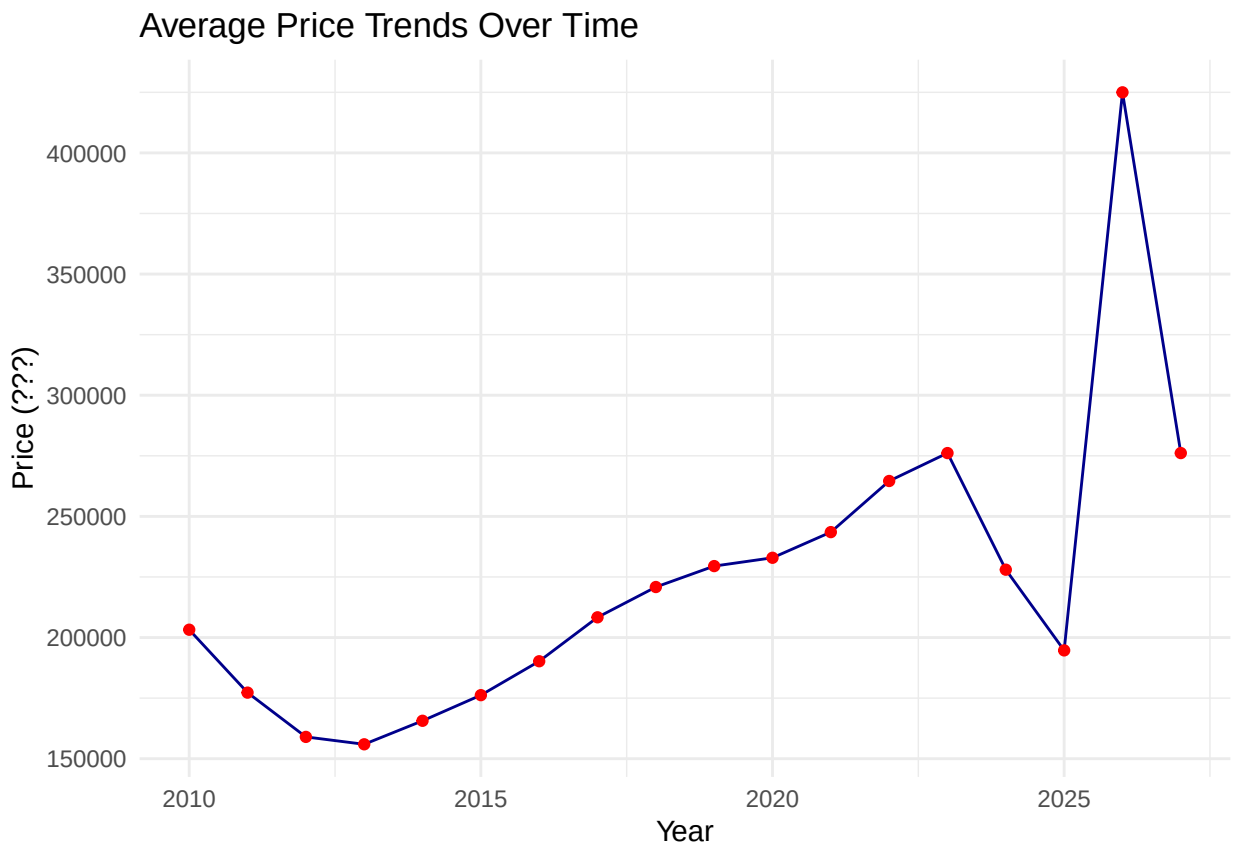

```
# Boxplot for price by property category (High/Low)
ggplot(combined_data_2b_clean, aes(x = price_category, y = price)) +
  geom_boxplot(fill = c("lightgreen", "lightcoral")) +
  labs(title = "Price by Property Category",
       x = "Price Category", y = "Price (€)")
```




```
# Ensure `description_length` is calculated correctly
combined_data_2b_clean <- combined_data_2b_clean %>%
  mutate(description_length = nchar(as.character(property_description))) %>%
  filter(price > 0 & description_length > 0) # Remove invalid values

# Scatter plot of price vs. description length
ggplot(combined_data_2b_clean, aes(x = description_length, y = price)) +
  geom_point(color = "purple", alpha = 0.5) +
  labs(title = "Price vs Description Length",
       x = "Description Length (characters)", y = "Price (???)") +
  theme_minimal()
```




```
# Remove rows with missing or zero values in 'sale_year' or 'price'
combined_data_2b_clean <- combined_data_2b_clean %>%
  filter(!is.na(price) & price > 0 & !is.na(sale_year))

# Calculate the average price per year
combined_data_2b_clean_agg <- combined_data_2b_clean %>%
  group_by(sale_year) %>%
  summarise(avg_price = mean(price, na.rm = TRUE))

ggplot(combined_data_2b_clean_agg, aes(x = sale_year, y = avg_price)) +
  geom_line(group = 1, color = "darkblue") +
  geom_point(color = "red") +
  labs(title = "Average Price Trends Over Time",
       x = "Year", y = "Price (???)") +
  theme_minimal()
```




```
# New Feature: Description Length (number of characters)
combined_data_2b_clean$description_length <-
  nchar(combined_data_2b_clean$property_description)

# New Feature: Price Category (High/Low based on price threshold)
price_threshold <- median(combined_data_2b_clean$price, na.rm = TRUE)
combined_data_2b_clean$price_category <-
  ifelse(combined_data_2b_clean$price > price_threshold, "High", "Low")
```

Data visualization plays a key role in identifying patterns, trends, and distributions within a dataset, which helps in understanding the underlying data and guiding further analysis. In this analysis of property data, several visualizations were generated to explore the dataset. The histogram of property prices provides an overview of the distribution, highlighting any skewness or outliers. The boxplot, which divides properties into "High" and "Low" price categories, reveals variations in price ranges within each group. The scatter plot of price versus description length helps determine whether longer property descriptions tend to correlate with higher prices. Finally, the line plot tracks price trends over time, showing any significant fluctuations or patterns in property prices.

In addition to these visualizations, new features were created to add depth to the dataset. The description length feature, which calculates the number of characters in the property description, was added to analyze if more detailed descriptions are associated with higher property prices. A price category feature was also created, categorizing properties as either "High" or "Low" based on their price relative to the median price. These new features not only provide additional insights into the dataset but also support feature selection and enhance predictive modeling efforts.

8 Conclusion

In conclusion, R Studio is an essential tool for working with property datasets. Its flexibility allows users to easily import and export data in various formats, like CSV and Excel, while built-in cleaning functions such as `na.omit()` and `mutate()` ensure data accuracy and consistency. With property datasets like "PropertyDataset1.csv" and "PropertyDataset2.csv," R Studio makes it easy to manage missing values, remove outliers, and create new variables (like property description length), providing valuable insights. These features simplify the data preparation process, ensuring it's ready for analysis.

R Studio also excels in data visualization, with packages like `ggplot2` allowing users to create clear and customizable plots. These visualizations make it easier to spot trends and relationships, such as the connection between property prices and factors like sale year or description length. Additionally, R Studio offers powerful statistical tools that enable users to apply advanced techniques like regression analysis to better understand what influences property prices. Overall, R Studio's intuitive interface and robust features make it a go-to platform for property data analysis, combining efficiency with detailed insights.